# Shiv Nadar University Chennai

## CS3807 – Deep Learning Laboratory

## Experiment 4

## Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

| | | |
|---|---|---|
| Degree & Branch | B.Tech Artificial Intelligence & Data Science | Semester V |
| Subject Code & Name | CS3807 – Deep Learning Laboratory | AY: 2026–27 |

## 1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

## 2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

## 3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

## 4. Evolution of CNN Architectures

| Model | Year | Major Contribution |
|---|---|---|
| LeNet-5 | 1998 | First practical convolutional neural network |
| AlexNet | 2012 | ReLU activation, Dropout and GPU training |
| VGG16 | 2014 | Uniform $3 \times 3$ convolution filters |
| GoogleNet | 2014 | Inception modules for multi-scale feature extraction |
| ResNet | 2015 | Residual learning using skip connections |

## 5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

## 6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

## 7. VGG16

VGG16 demonstrated that using multiple small $3 \times 3$ convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

## 8. Comparison of Architectures

| Architecture | Depth | Parameters | Main Contribution |
|---|---|---|---|
| LeNet-5 | 7 | 60K | First CNN |
| AlexNet | 8 | 61M | ReLU + Dropout |
| VGG16 | 16 | 138M | Deep 3×3 filters |
| GoogleNet | 22 | 6.8M | Inception Modules |
| ResNet50 | 50 | 25.6M | Residual Learning |

## 9. GoogleNet (Inception Network)

GoogleNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogleNet simultaneously applies $1 \times 1$, $3 \times 3$, $5 \times 5$ convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of $1 \times 1$ convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

## 10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- $x$ = Identity Mapping

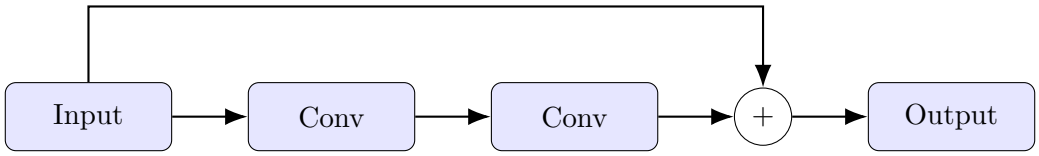


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

## 11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis

- Satellite imagery
- Object localization

## 12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

## 13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

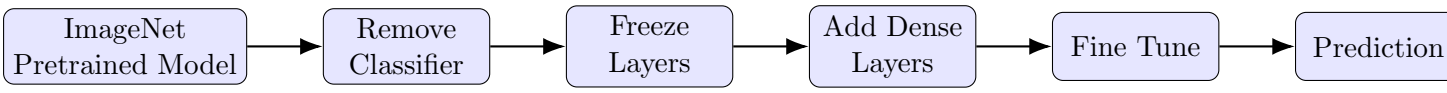


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

## 14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size : $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

## 15. Experimental Procedure

### Task 1: Dataset Preparation

1. Load the CIFAR-10 dataset using TensorFlow/Keras.
2. Normalize the pixel values to the range [0,1].
3. Display ten sample images.

4. Print the dimensions of the training and testing datasets.

### Task 2: Transfer Learning

Load any one of the following pretrained CNN models.

- VGG16
- ResNet50
- MobileNetV2
- EfficientNetB0 (Optional)

Perform the following steps.

1. Load pretrained ImageNet weights.
2. Remove the original classification layer.
3. Freeze the convolutional base.
4. Add a Global Average Pooling layer.
5. Add a Dense layer with ReLU activation.
6. Add the output layer with Softmax activation.

### Task 3: Model Training

Train the model using

| | |
|---|---|
| Optimizer | Adam |
| Learning Rate | 0.001 |
| Batch Size | 32 |
| Epochs | 10–20 |
| Loss Function | Categorical Cross Entropy |
| Evaluation Metric | Accuracy |

### Task 4: Fine Tuning

1. Unfreeze the last convolution block.
2. Train for another 5–10 epochs.
3. Compare the classification accuracy before and after fine tuning.

### Task 5: Model Evaluation

Evaluate the trained model using

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

## 16. Hyperparameter Study

Compare the performance using different settings.

| Hyperparameter | Values |
|---|---|
| Learning Rate | 0.001, 0.0001 |
| Batch Size | 16, 32, 64 |
| Epochs | 10, 20 |
| Optimizer | Adam, SGD |
| Dense Units | 128, 256 |
| Frozen Layers | All, Partial |

## 17. Mandatory Plots

Include the following plots in the laboratory report.

1. Sample CIFAR-10 images
2. Training Accuracy
3. Validation Accuracy
4. Training Loss
5. Validation Loss
6. Confusion Matrix
7. Misclassified Images (Optional)

Students should write a brief inference (2–3 lines) for each figure.

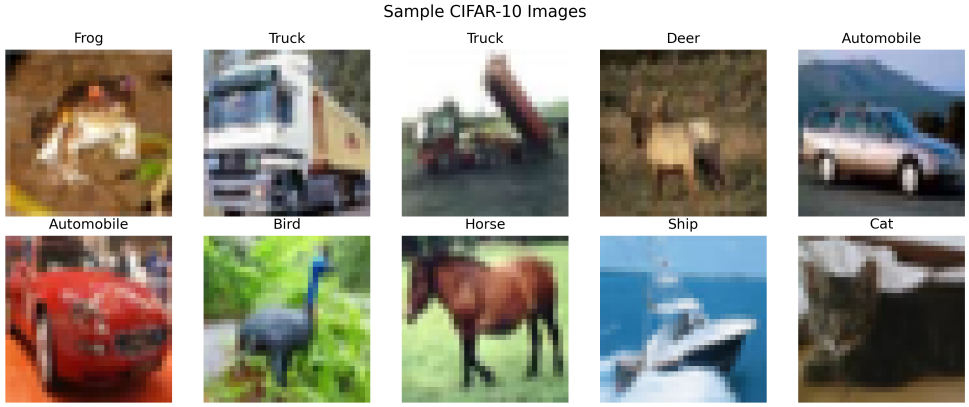


Figure 3: Sample CIFAR-10 Images

*Inference:* The figure shows ten representative $32 \times 32 \times 3$ images from the CIFAR-10 training set spanning all ten classes. The low resolution and visual similarity between certain classes (e.g., cat vs. dog, automobile vs. truck) illustrate why CIFAR-10 is a non-trivial classification benchmark even for deep pretrained models.


Figure 4: Training vs Validation Accuracy – ResNet50 (Initial Training, Frozen Base)

*Inference:* During the initial 15-epoch training phase with the ResNet50 base frozen, training accuracy rose steadily to 93.75%, while validation accuracy plateaued near 64%. The widening gap between the curves indicates the frozen convolutional base combined with the classifier head begins to overfit the training data past roughly epoch 5.

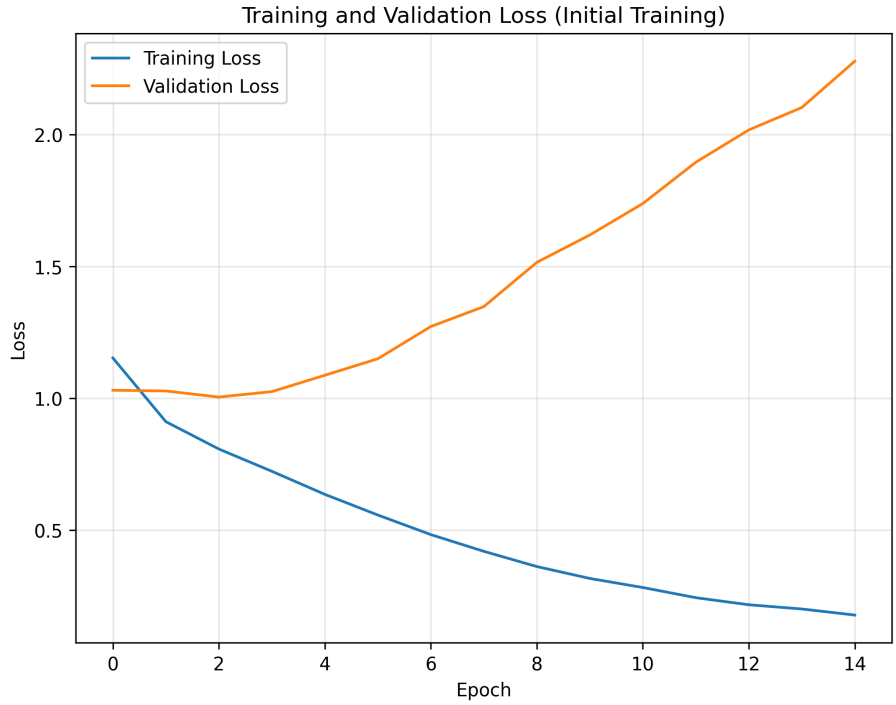


Figure 5: Training vs Validation Loss – ResNet50 (Initial Training, Frozen Base)

*Inference:* Training loss decreased consistently from 1.15 to 0.18 across the 15 epochs, but validation loss increased steadily after epoch 3 (from about 1.00 to 2.28). This confirms the overfitting trend seen in the accuracy curves and motivates the fine-tuning stage.

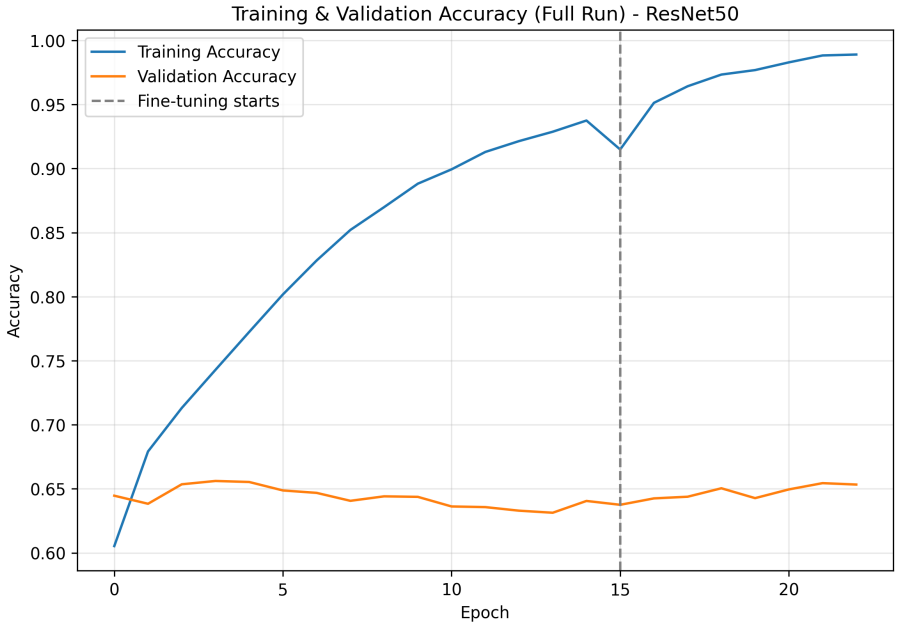


Figure 6: Combined Training and Validation Accuracy – Initial Training + Fine-Tuning (ResNet50)

*Inference:* The dashed line marks the start of fine-tuning. Validation accuracy improves from 64.05% to a final 65.33% once the last convolution block is unfrozen, while training accuracy climbs further to 98.90%, showing the fine-tuned layers fit the training set more aggressively than they generalize.

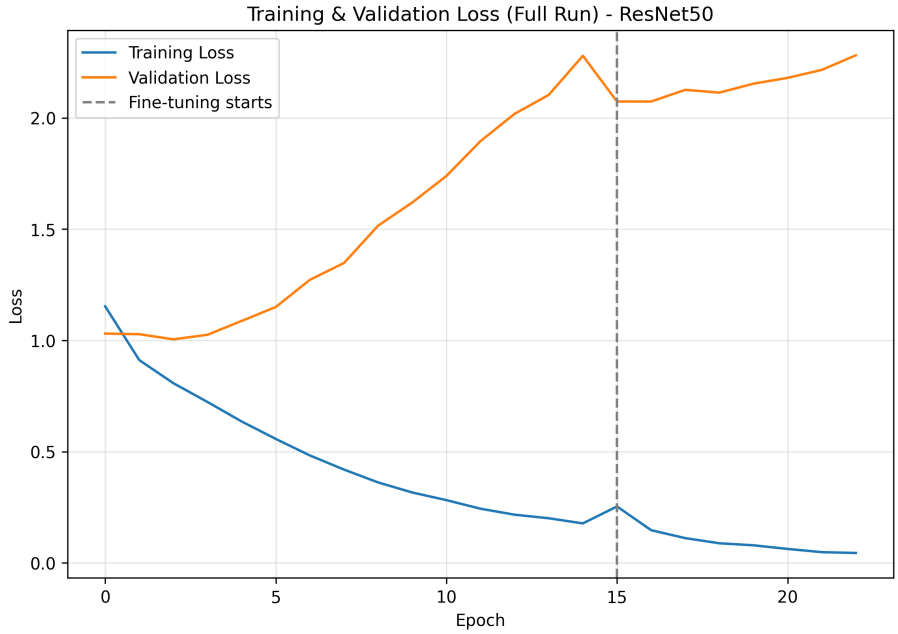


Figure 7: Combined Training and Validation Loss – Initial Training + Fine-Tuning (ResNet50)

*Inference:* Training loss continues to fall sharply during fine-tuning (down to 0.04), while validation loss keeps rising past the point where fine-tuning begins. This gap is the clearest evidence of overfitting once the deeper layers are unfrozen with a small CIFAR-10 image size.

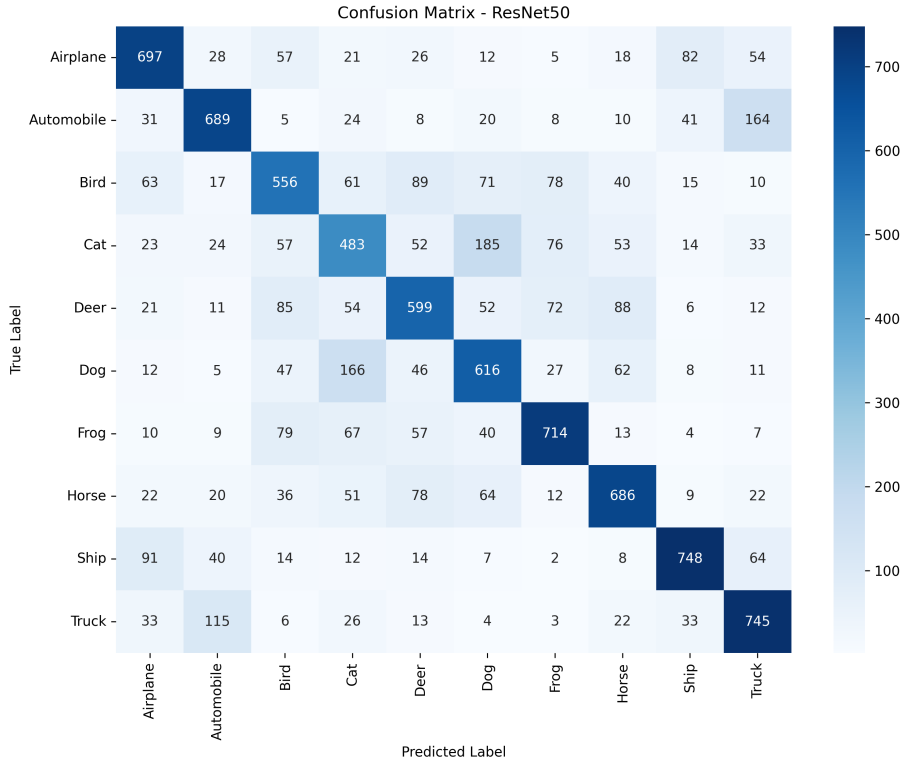


Figure 8: Confusion Matrix – ResNet50

*Inference:* ResNet50 performs best on the Ship (76% F1) and Frog (72% F1) classes, while Cat (49% F1) and Bird (57% F1) show the most confusion, frequently being mistaken for visually similar animal classes such as Dog and Deer.


Figure 9: Misclassified Images – ResNet50 (Optional)

*Inference:* 3,467 of the 10,000 test images (34.67%) were misclassified by ResNet50. The sampled misclassifications mostly occur between visually or semantically close categories (e.g., cat/dog, automobile/truck), consistent with the per-class precision and recall values in the classification report.

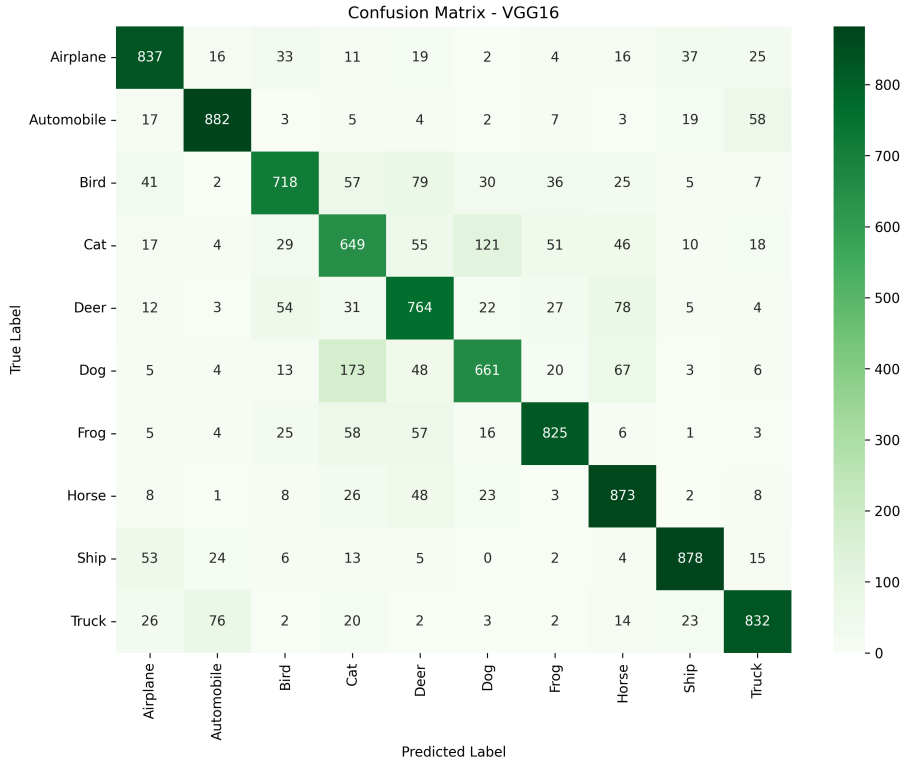


Figure 10: Confusion Matrix – VGG16

*Inference:* VGG16 reaches 79.19% test accuracy with more balanced per-class performance than ResNet50, benefiting from full fine-tuning of its deep $3 \times 3$ filter stack even at the small $32 \times 32$ input size.

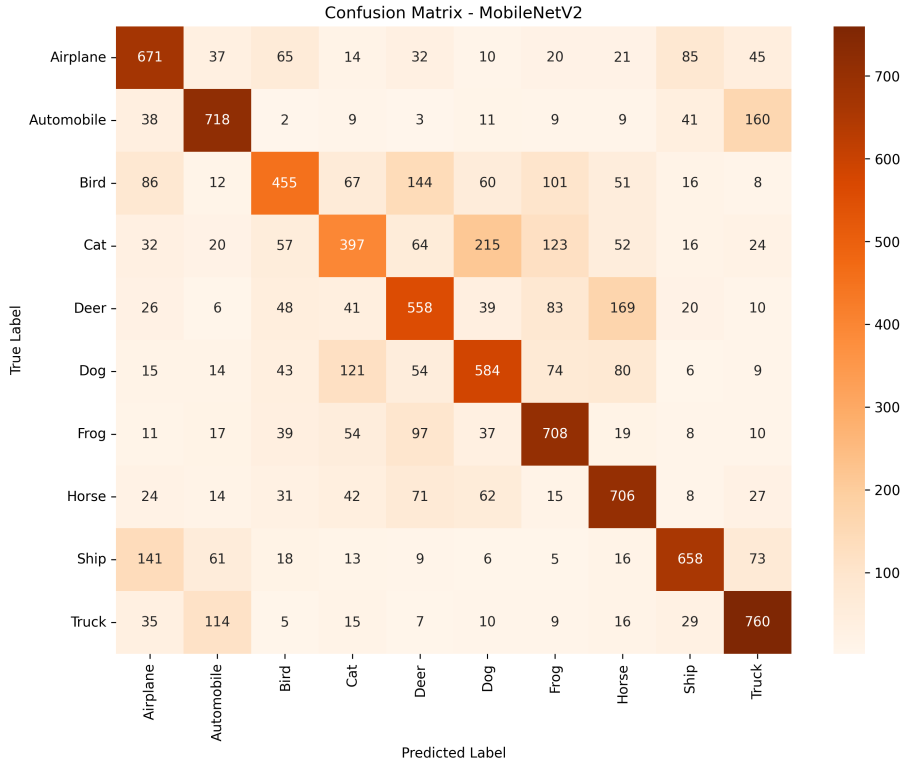


Figure 11: Confusion Matrix – MobileNetV2

*Inference:* MobileNetV2 shows the weakest performance among the transfer-learning models at 62.15% accuracy. Its lightweight depthwise-separable design, pretrained at $224 \times 224$, adapts less effectively to

the very small $32 \times 32$ CIFAR-10 input than the larger ResNet50 and VGG16 backbones.


Figure 12: Confusion Matrix – GoogleNet (InceptionV3)

*Inference:* InceptionV3, evaluated after resizing CIFAR-10 images to $75 \times 75$, reaches 70.04% test accuracy. Its multi-scale Inception filters help, but up-sampling from a $32 \times 32$ source introduces interpolation artifacts that likely limit further gains.

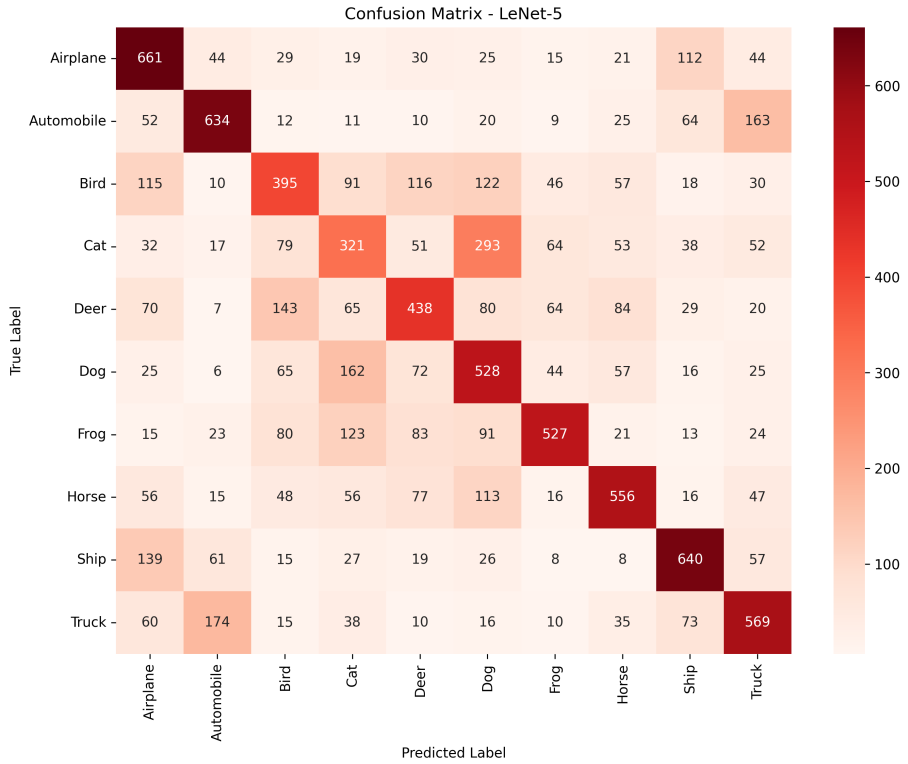


Figure 13: Confusion Matrix – LeNet-5

*Inference:* As a shallow architecture trained entirely from scratch (no pretrained weights are available for LeNet-5), it achieves the lowest accuracy overall at 52.69%, reflecting its limited capacity to capture complex features in a 10-class natural image dataset.

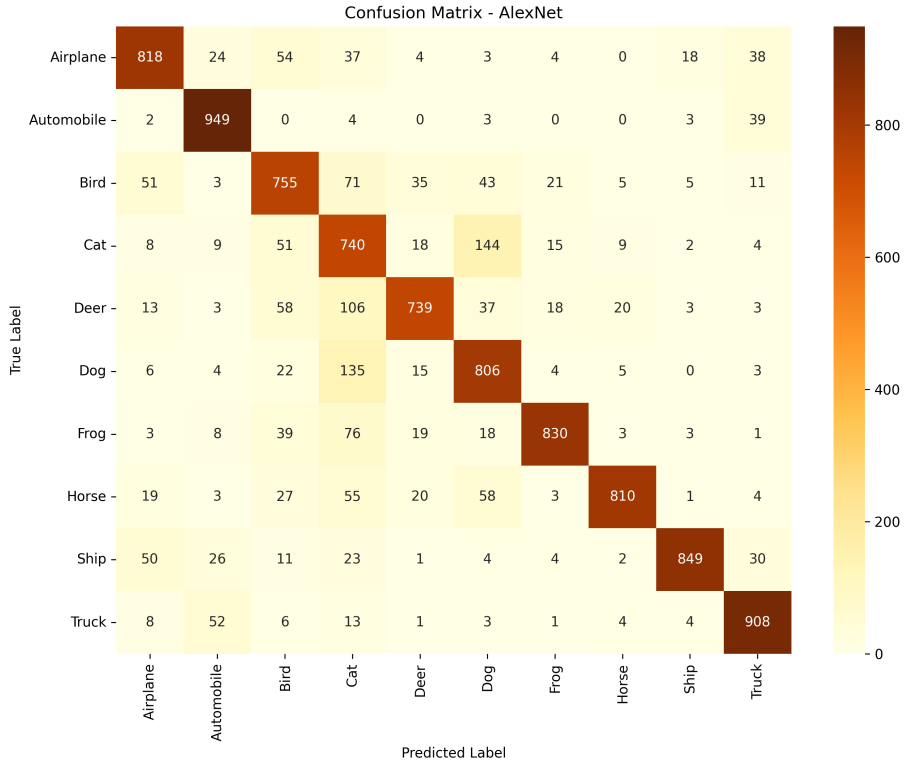


Figure 14: Confusion Matrix – AlexNet

*Inference:* AlexNet, trained from scratch with Batch Normalization and Dropout, achieves the highest overall test accuracy (82.04%) despite having no pretrained weights, showing that a well-regularized architecture matched to CIFAR-10's scale can outperform ImageNet-pretrained backbones whose native resolution mismatch limits transfer effectiveness.

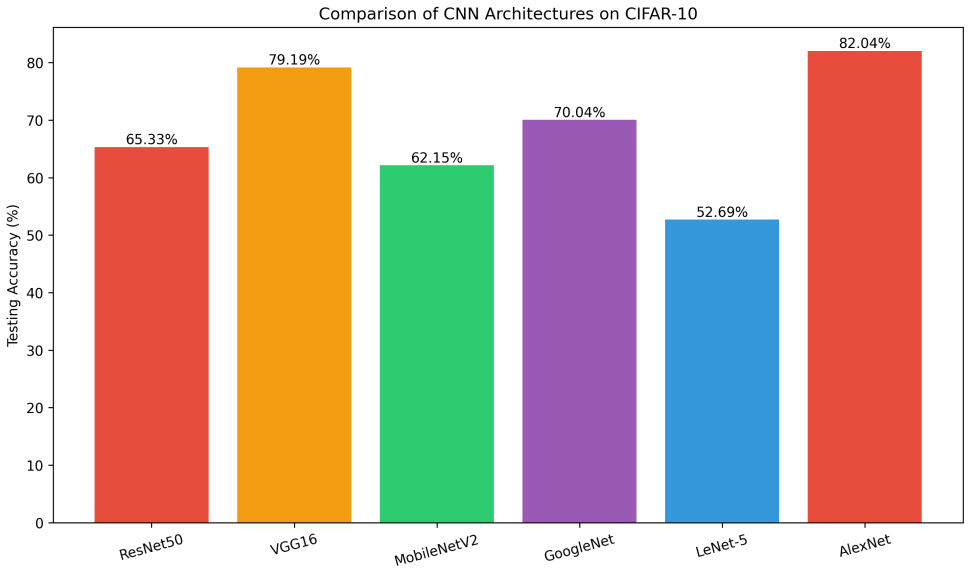


Figure 15: Testing Accuracy Comparison Across All Five CNN Architectures

*Inference:* AlexNet (82.04%) and VGG16 (79.19%) achieved the highest test accuracy on CIFAR-10, outperforming the deeper ResNet50 (65.33%) and InceptionV3 (70.04%). This suggests that, for a small

$32 \times 32$ dataset with limited fine-tuning epochs, architectures adapted/trained closer to the target input scale can outperform larger ImageNet-pretrained backbones whose learned features are optimized for $224 \times 224$ inputs.

# 18. Results

## 18.1 Performance Metrics

| Metric | Value |
|---|---|
| Training Accuracy | 98.90% |
| Testing Accuracy | 65.33% |
| Precision | 65.35% |
| Recall | 65.33% |
| F1-score | 65.30% |
| Training Time | 6.18 minutes |
| Total Parameters | 24,114,826 |

## 18.2 Comparison of CNN Architectures

| Model | Parameters | Accuracy (%) | Training Time |
|---|---|---|---|
| LeNet-5 | 83,126 | 52.69 | 1.83 min |
| AlexNet | 5,282,378 | 82.04 | 4.87 min |
| VGG16 | 14,848,586 | 79.19 | 6.49 min |
| GoogleNet | 22,329,898 | 70.04 | 11.15 min |
| ResNet50 | 24,114,826 | 65.33 | 6.18 min |

# 19. Discussion Questions

Answer the following questions.

1. Why is AlexNet considered a breakthrough in deep learning?
   *Answer:* AlexNet (2012) was the first deep CNN to win ImageNet by a large margin. It proved that deep networks could be trained efficiently on GPUs, and it popularised ReLU activation (faster convergence than sigmoid/tanh), Dropout (reduces overfitting), and data augmentation – ideas that became standard in nearly all later architectures.
2. Why does VGG16 use only $3 \times 3$ convolution filters?
   *Answer:* Stacking multiple $3 \times 3$ filters gives the same effective receptive field as a larger filter (e.g. two $3 \times 3$ layers $\approx$ one $5 \times 5$) while using fewer parameters and adding an extra non-linearity (ReLU) between them, which improves feature discrimination and keeps the network simpler to design.
3. Explain the advantages of the Inception module.
   *Answer:* The Inception module applies multiple filter sizes ($1 \times 1$, $3 \times 3$, $5 \times 5$) and pooling in parallel and concatenates their outputs, capturing features at multiple scales in a single layer. The $1 \times 1$ convolutions before larger filters reduce dimensionality first, cutting computation and parameters while keeping accuracy high.
4. What is the purpose of residual learning?
   *Answer:* Residual learning lets a layer learn only the residual $F(x) = H(x) - x$ instead of the full mapping $H(x)$, with the output computed as $F(x) + x$ via a skip connection. This allows

gradients to flow directly through the identity path, solving the vanishing-gradient problem and enabling much deeper networks to be trained successfully.

5. Differentiate LeNet and ResNet.
   *Answer:* LeNet-5 (1998) is a shallow 7-layer network with ∼60K parameters, using average pooling and tanh activations, designed for small grayscale digit images. ResNet50 (2015) is a 50-layer deep network with ∼25.6M parameters, using residual/skip connections and ReLU, capable of learning much richer hierarchical features from large-scale natural images. In our experiment this showed up directly: LeNet-5 reached only 52.69% test accuracy on CIFAR-10 versus ResNet50's much larger capacity, though ResNet50's transfer-learning accuracy (65.33%) was still limited here by the small $32 \times 32$ input.
6. What is Transfer Learning?
   *Answer:* Transfer learning reuses a CNN already pretrained on a large dataset (e.g. ImageNet) for a new, related task. Instead of training from scratch, the pretrained convolutional base (which has already learned general features like edges and textures) is reused, its classifier head is replaced, and only the new layers (or a few unfrozen top layers) are trained on the new dataset – reducing training time and data requirements.
7. Why is fine tuning required?
   *Answer:* With only the classifier head trained and the base frozen, the pretrained features may not be fully specialised to the new dataset. Fine tuning unfreezes the last few convolutional layers and trains them at a low learning rate, letting the network adapt its high-level features to the target data and typically improving accuracy further – as seen in our ResNet50 run, where validation accuracy rose from 64.05% to 65.33% after fine tuning.
8. Explain the difference between dilated convolution and transpose convolution.
   *Answer:* Dilated (atrous) convolution inserts gaps between kernel elements to enlarge the receptive field without adding parameters or reducing spatial resolution – useful for tasks like semantic segmentation. Transpose convolution does the opposite job: it performs learnable upsampling to increase the spatial size of a feature map, commonly used in autoencoders, GANs, and image generation.
9. Why do pretrained models converge faster?
   *Answer:* Pretrained models start with weights that already encode general-purpose visual features (edges, textures, shapes) learned from a large dataset like ImageNet, rather than random weights. This gives training a strong starting point, so fewer epochs are needed to reach good performance compared to training from scratch. This was visible in our results: the transfer-learning models (ResNet50, VGG16, MobileNetV2, GoogleNet) reached high training accuracy within just a few epochs, though full generalisation to CIFAR-10's small $32 \times 32$ images was still limited.
10. Compare the computational complexity of LeNet and ResNet.
    *Answer:* LeNet-5 is extremely lightweight (∼60K–83K parameters in our implementation), with only two convolution layers, making it fast to train but limited in representational power. ResNet50 is far more complex (24.1M parameters, 50 layers), requiring significantly more computation and memory per forward/backward pass. This matched our timing results: LeNet-5 trained in 1.83 minutes versus ResNet50's 6.18 minutes for a comparable number of epochs.

## 20. Additional Exercises

1. Implement transfer learning using VGG16.
2. Repeat the experiment using ResNet50.
3. Compare Adam and SGD optimizers.
4. Compare training with frozen layers and fine tuning.
5. Evaluate the model using Precision, Recall and F1-score.
6. Compare the performance of LeNet, AlexNet and ResNet.

## 21. References


1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: `https://www.tensorflow.org`
8. Keras Documentation: `https://keras.io`